

Sparse MoE: Routing & Load Balancing.

A learned router sends each token to a few of E experts. This buys huge parameter count at a fixed compute budget — but only if the router spreads load evenly. We derive top- k routing, the capacity bound, and the load-balancing loss that prevents expert collapse. Every number on the following pages is computed in full — nothing summarised or deferred.

THE EQUATION

$$y = \sum_{i \in \text{Top}_k(g)} \text{softmax}(g)_i \cdot \text{Expert}_i(x), \quad g = x W_g$$

Worked example. $E=8$ experts, top-2, a batch of 6 tokens: full routing matrix, one token's gated output, f_i , P_i , and $\mathcal{L}_{\text{aux}}$.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	The router and the sparse mixture	2
2	The routing matrix (TikZ)	3
3	Worked example: top-2 over 8 experts	3
4	Capacity and overflow	3
5	The load-balancing loss $\mathcal{L}_{\text{aux}}$	4
6	Mixtral 8x7B: the parameter arithmetic	5
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The router and the sparse mixture

A Mixture-of-Experts (MoE) layer holds E separate feed-forward experts. For a token embedding $x \in \mathbb{R}^d$, a linear *router* $W_g \in \mathbb{R}^{d \times E}$ produces gate logits $g = xW_g \in \mathbb{R}^E$. A softmax turns them into routing probabilities, and only the top- k experts are actually evaluated — the rest contribute nothing, so compute stays $O(k)$ experts regardless of E .

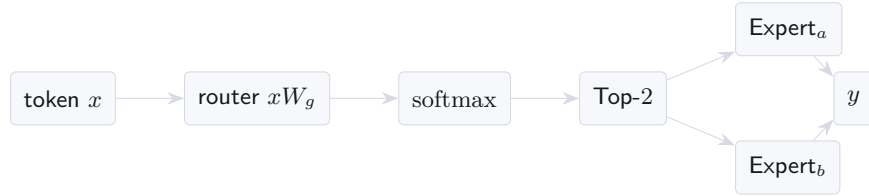
$$g = xW_g, \quad p = \text{softmax}(g), \quad y = \sum_{i \in \text{Top}_k(g)} \frac{\exp(g_i)}{\sum_{j \in \text{Top}_k(g)} \exp(g_j)} \text{Expert}_i(x) \quad (1)$$

The gate weights are the softmax *re-normalised over the selected experts* (so they sum to 1).

Symbol	Meaning	Shape
x	token embedding	$\mathbb{R}^d$
W_g	router weights	$\mathbb{R}^{d \times E}$
g, p	gate logits, softmax probs	$\mathbb{R}^E$
E, k	#experts, #selected (top- k)	scalar
Expert_i	i -th feed-forward network	$\mathbb{R}^d \rightarrow \mathbb{R}^d$
f_i, P_i	routed fraction, mean prob for expert i	scalar

The router is a tiny classifier over experts. “Sparse” means each token wakes up only k of the E specialists; the model can store the knowledge of all E experts but pays the FLOPs of only k . Capacity grows with E , compute grows with k .

2 The routing matrix (TikZ)



3 Worked example: top-2 over 8 experts

Take $E = 8$, $k = 2$, and a batch of $N = 6$ tokens with router logits g (rows = tokens). Applying a row-wise softmax gives the routing-probability matrix $P = \text{softmax}(g)$:

	e1	e2	e3	e4	e5	e6	e7	e8
t1	.437	.072	.032	.217	.059	.020	.098	.065
t2	.059	.321	.072	.036	.355	.053	.026	.079
t3	.033	.081	.443	.066	.049	.243	.060	.024
t4	.230	.063	.057	.380	.031	.077	.069	.094
t5	.071	.286	.064	.078	.086	.058	.316	.043
t6	.094	.063	.231	.057	.070	.077	.063	.345

The two bold cells per row are the top-2 picks. Reading them off:

$$t1 \rightarrow \{1, 4\}, t2 \rightarrow \{5, 2\}, t3 \rightarrow \{3, 6\}, t4 \rightarrow \{4, 1\}, t5 \rightarrow \{2, 7\}, t6 \rightarrow \{8, 3\}.$$

3.1 Gated output for one token

For token t1, the selected logits are $g = (2.1, 1.4)$ for experts $\{1, 4\}$. The re-normalised gate weights (softmax over just those two) are

$$w_1 = \frac{e^{2.1}}{e^{2.1} + e^{1.4}} = \frac{8.166}{8.166 + 4.055} = 0.6682, \quad w_4 = \frac{e^{1.4}}{e^{2.1} + e^{1.4}} = 0.3318,$$

identical to dividing the full-softmax probabilities 0.437, 0.217 by their sum 0.654. Hence

$$y_{t1} = 0.6682 \cdot \text{Expert}_1(x) + 0.3318 \cdot \text{Expert}_4(x)$$

and experts 2, 3, 5, 6, 7, 8 are never evaluated for this token.

4 Capacity and overflow

Each expert can process at most a fixed number of tokens per batch:

$$C = \text{capacity_factor} \cdot \frac{\#\text{token-slots}}{E}, \quad \#\text{slots} = N \cdot k.$$

With $N=4096$, $k=2$, $E=8$, $\text{capacity_factor} = 1.25$:

$$C = 1.25 \cdot \frac{4096 \cdot 2}{8} = 1.25 \cdot 1024 = \mathbf{1280} \text{ tokens/expert.}$$

A token routed to an expert that is already full *overflows* and is dropped (its residual passes through unchanged). The factor 1.25 leaves 25% headroom above the uniform mean 1024.

Capacity is a per-batch budget, not a per-token one. The total slots $N \cdot k$ are split evenly across E experts and inflated by the capacity factor. If routing is balanced, almost nothing overflows; if it is skewed, the hot experts saturate and tokens are dropped — which is exactly what the load-balancing loss is designed to prevent.

5 The load-balancing loss $\mathcal{L}_{\text{aux}}$

Left alone, the router collapses: a few experts win, the rest starve. The auxiliary loss penalises imbalance using two per-expert statistics over the batch,

$$f_i = \frac{\#\{\text{tokens with } i \in \text{Top}_k\}}{N}, \quad P_i = \frac{1}{N} \sum_t p_{t,i},$$

the routed *fraction* and the mean router *probability*. The loss is

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^E f_i P_i \quad (2)$$

The product $f_i P_i$ is large only when an expert is both chosen often *and* confidently; the E factor scales it so the uniform optimum is 1. Minimising it (subject to $\sum f_i = k$, $\sum P_i = 1$) drives f and P toward uniform. Computing from our batch ($N=6$):

expert i	1	2	3	4	5	6	7	8
count (top-2)	2	2	2	2	1	1	1	1
$f_i = \text{count}/6$	.333	.333	.333	.333	.167	.167	.167	.167
P_i (mean prob)	.154	.148	.150	.139	.108	.088	.105	.108
$f_i P_i$	.051	.049	.050	.046	.018	.015	.018	.018

Summing, $\sum_i f_i P_i = 0.2650$, so $E \sum f_i P_i = 8 \cdot 0.2650 = 2.1202$, and with $\alpha = 0.01$:

$$\mathcal{L}_{\text{aux}} = 0.01 \cdot 8 \cdot 0.2650 = \mathbf{0.02120}$$

Perfect balance would give $f_i = k/E = 0.25$, $P_i = 1/E = 0.125$, hence $E \sum f_i P_i = 8 \cdot 8 \cdot (0.25 \cdot 0.125)/8 = 1.0$ — our 2.12 flags the imbalance (experts 1–4 carry twice the load of 5–8).

$\mathcal{L}_{\text{aux}} = \alpha E \sum_i f_i P_i$ is minimised, at value 1, exactly when both the routed fraction f and the mean probability P are uniform. Using f (a hard count, non-differentiable) times P

(a soft, differentiable probability) lets gradients flow into the router while still measuring the true dispatch.

6 Mixtral 8x7B: the parameter arithmetic

Mixtral 8x7B uses $E = 8$ experts with top- $k = 2$ routing per token. Only $k/E = 2/8 = \mathbf{25\%}$ of the expert FLOPs fire per token. Because attention and embeddings are shared, the active parameter count is not simply $2/8$ of the total: $\sim 13\text{B}$ of the $\sim 47\text{B}$ parameters are active per token,

$$\frac{13}{47} = 0.2766 = \mathbf{27.7\%} \text{ active.}$$

So Mixtral has the knowledge capacity of a 47B model at roughly the inference cost of a 13B one.

Without $\mathcal{L}_{\text{aux}}$, MoE training suffers *expert collapse*: the router keeps picking the same handful of experts, they get all the gradient, become better, and get picked even more — a winner-take-all feedback loop. The starved experts stay random and waste parameters, while the popular ones blow past capacity C and start dropping tokens. The auxiliary loss is not optional polish; it is what makes sparse routing trainable at all.

7 Where this sits in the track

Back	Module 3.2 T1 — sliding-window attention (the other half of Mistral’s efficiency story).
This	Module 3.2 T2 — sparse MoE: top- k routing, capacity, load balancing (Mixtral).
Next	Phase 3 wrap-up — combining SWA + MoE into a full modern decoder block.

A Reproducible (NumPy)

Inputs: logits g (6 tokens $\times$ 8 experts), $k=2$, $\alpha=0.01$. Verified outputs: top-2 picks as listed; t1 gate weights (0.6682, 0.3318); $\sum f_i P_i = 0.2650$; $\mathcal{L}_{\text{aux}} = 0.02120$; capacity $C = 1280$ at $N=4096$; Mixtral active = 27.7%.

```
import numpy as np
g = np.array([
    [2.1, 0.3,-0.5, 1.4, 0.1,-1.0, 0.6, 0.2],
    [0.2, 1.9, 0.4,-0.3, 2.0, 0.1,-0.6, 0.5],
    [-0.4,0.5, 2.2, 0.3, 0.0, 1.6, 0.2,-0.7],
    [1.5, 0.2, 0.1, 2.0,-0.5, 0.4, 0.3, 0.6],
    [0.3, 1.7, 0.2, 0.4, 0.5, 0.1, 1.8,-0.2],
    [0.6, 0.2, 1.5, 0.1, 0.3, 0.4, 0.2, 1.9]])
P = np.exp(g - g.max(1, keepdims=True)); P /= P.sum(1, keepdims=True)
N, E, k, alpha = P.shape[0], 8, 2, 0.01
top = np.argsort(-P, 1)[:,:k] # top-2 expert indices
w0 = np.exp(g[0, top[0]]); w0 /= w0.sum() # t1 gate weights
f = np.bincount(top.ravel(), minlength=E) / N # routed fraction
Pi = P.mean(0) # mean router prob
```

```
L = alpha * E * (f * Pi).sum()
print(w0)                # [0.6682 0.3318]
print((f*Pi).sum(), L)    # 0.26502... 0.021201...
print(1.25 * N_k for N_k in [4096*2/8])    # capacity 1280 at N=4096
print(13/47)              # 0.27659... active
```

Marevlo Research · Transformer Track · Module 3.2 · Topic 2 of 2.